MMA3001 - NUMERICAL METHODS AND MACHINE LEARNING

FROZEN-MEAT PACKAGING QUALITY DETECTION USING YOLO26 OBJECT DETECTION

Student Name: _____
Student ID: _____

Unit: MMA3001
Project Type: Individual Project

2026

1. Engineering Problem

The project addresses automated visual inspection of frozen-meat packaging, using pork-rasher images as the available case study. In a packaging line, visible errors in product placement or sealing can indicate process inconsistency and can require manual inspection or rework. The computational task is therefore to detect and localise visible defect classes in RGB images so that inspection can be performed more consistently and at a higher throughput than purely manual checking.

The scope was deliberately narrowed to a five-class object-detection problem: loose-meat, packaging-error, twisted-meat, unsealed, and wrinkle. This is an achievable subset of the wider “frozen-meat packaging quality” topic in the project brief. The system does not attempt to determine food safety, shelf life, physical seal strength, or a complete industrial pass/fail policy. Its intended use is as a machine-vision aid that highlights labelled visual defects for later review or integration into a larger quality-control process.

2. Inputs, Outputs and Data

The primary input is an RGB image of a packaged pork-rasher product. During training and evaluation, Ultralytics resizes images to the selected model input size. The main experiment uses 640 pixels. Object annotations are stored in YOLO text format as a class identifier and normalised bounding-box coordinates (x-centre, y-centre, width, height).

The model output is a set of predicted bounding boxes, class labels, and confidence scores. These outputs identify where a visible defect is located and what defect type the model predicts. Aggregate outputs used for engineering evaluation are Precision, Recall, mAP@50, mAP@50-95, inference time, model-comparison tables, confusion matrices, and example prediction images.

Split	Images	Empty labels	loose	packaging	twisted	unsealed	wrinkle
Train	3349	684	66	1712	72	1055	18
Validation	120	14	3	34	3	81	0
Test	80	12	3	23	4	49	0

Table 1. Audited image and annotation counts.

Across all splits there are 3,549 images and 3,123 annotated objects. The dataset is strongly imbalanced: packaging-error contributes 1,769 annotations (56.6%) and unsealed 1,185 (37.9%), while wrinkle has only 18 annotations (0.6%). This imbalance is important when interpreting global metrics.

3. Computational Approach

A pretrained YOLO26 detector was selected because the problem requires both classification and localisation. A classifier alone could predict a defect label but would not indicate where the defect is located. Object detection therefore matches the available bounding-box annotations and the intended visual-inspection use.

Two model sizes were compared under the same main training settings. YOLO26n was used as the baseline because it is the smaller model. YOLO26s was used as the credible alternative because it increases model capacity without changing the overall detection method. This keeps the comparison simple: the main changed factor is model size.

Setting	**Value**
Baseline model	YOLO26n
Alternative model	YOLO26s
Pretrained weights	Yes
Maximum epochs	30
Input image size	640
Batch size	16
Random seed	42
Early-stopping patience	10
Selection metric	Validation mAP@50-95
Hardware used	Tesla T4 GPU (Google Colab)

Table 2. Main training configuration.

The workflow was: (1) download the Roboflow dataset, (2) audit images and annotations, (3) run a one-epoch smoke test, (4) train YOLO26n, (5) validate YOLO26n, (6) train YOLO26s using the same settings, (7) compare both models on validation data, (8) select the model with the highest validation mAP@50-95, (9) perform a small input-resolution sensitivity check, and (10) evaluate the selected model once on the withheld test split. The test set was therefore not used to choose the model.

The implementation used Python, Jupyter/Google Colab, PyTorch, Ultralytics, Roboflow, pandas, OpenCV and matplotlib. Reusable data-check and model-selection functions are also provided as Python source files with docstrings and unit tests. A separate HTML API document and test report are generated for the repository deliverable.

4. Validation Method and Data Quality

Validation was based on predefined train, validation and test splits supplied with the dataset. The validation split was used during model comparison, while the test split was reserved for the final selected model. This separation reduces the risk of choosing a model based on its performance on the final evaluation data.

The main accuracy metrics were Precision, Recall, mAP@50 and mAP@50-95. Precision measures how many reported detections are correct, Recall measures how many labelled objects are found, mAP@50 summarises average precision at an intersection-over-union threshold of 0.50, and mAP@50-95 is the stricter COCO-style average across IoU thresholds from 0.50 to 0.95. The primary model-selection metric was validation mAP@50-95 because it rewards both correct classification and accurate localisation over several overlap thresholds.

Before training, the dataset was audited for image counts, label files, empty annotations and per-class support. An additional diagnostic check was added for missing labels, missing images, invalid YOLO rows, invalid class IDs and coordinates outside the normalised 0-1 range. The saved diagnostic file contains 710 entries, and all 710 are empty-label files: 684 in training, 14 in validation and 12 in test. No other diagnostic issue type appears in the saved output. Empty labels were not automatically deleted because they may represent valid images containing no labelled defect.

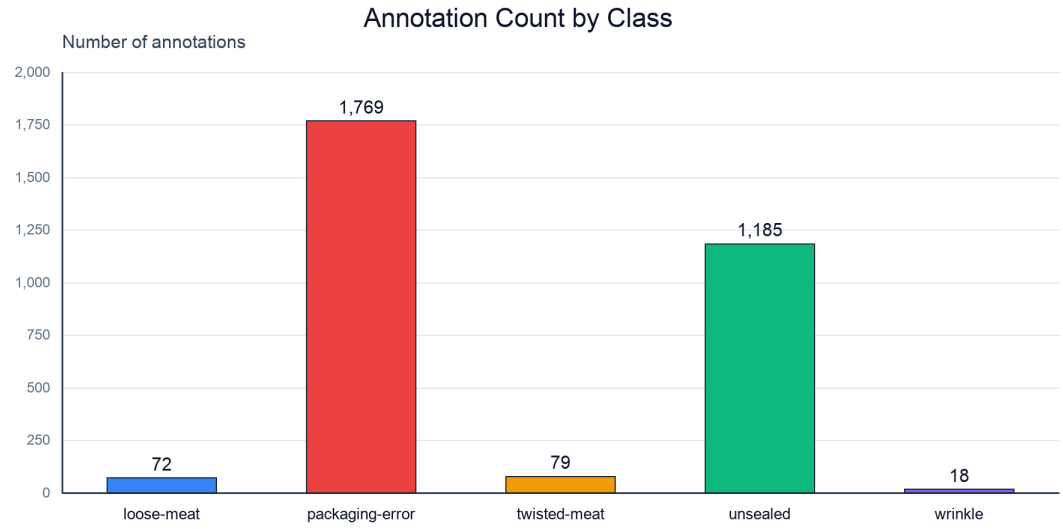


Figure 1. Total annotation count by class. The vertical scale shows the strong imbalance between common and rare classes.

A major validation limitation is class support. The validation split contains only three loose-meat instances and three twisted-meat instances, and the test split contains three and four respectively. The wrinkle class has zero instances in both validation and test. Therefore, a reliable validation claim cannot be made for wrinkle, and per-class conclusions for the rare classes must be treated cautiously.

5. Model Comparison and Selection

Table 3 compares the measured validation results. YOLO26n produced the highest Precision (0.8573), but its Recall was only 0.4685. YOLO26s reduced Precision to 0.6912 but increased Recall to 0.6972. It also achieved higher mAP@50 and mAP@50-95. Because mAP@50-95 was the predefined selection metric, YOLO26s was selected as the final model.

Model	Precision	Recall	mAP@50	mAP@50-95	Infer. ms	Train min
YOLO26n	0.8573	0.4685	0.5593	0.3638	12.47	28.50
YOLO26s	0.6912	0.6972	0.6587	0.3983	11.20	31.55

Table 3. Validation comparison of the baseline and alternative models.

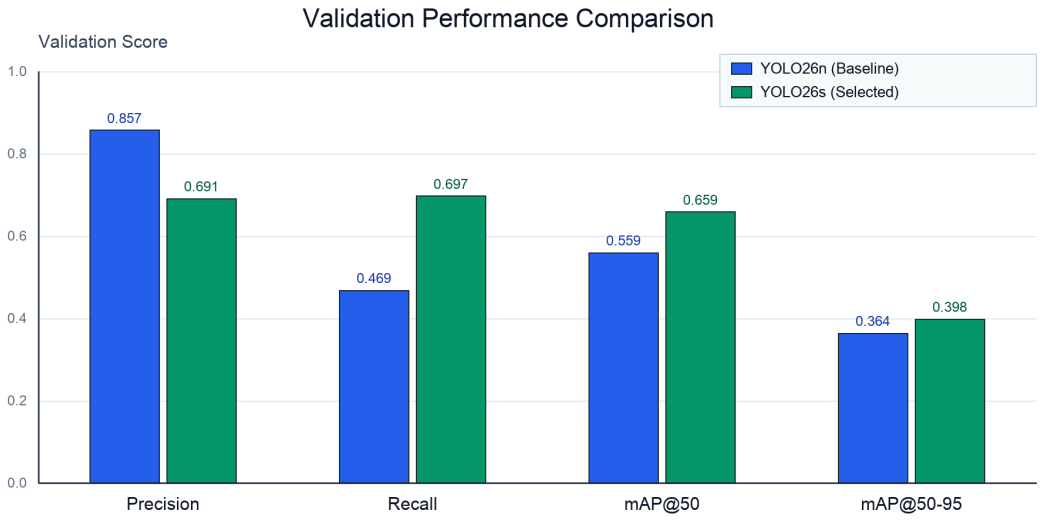


Figure 2. Validation metrics for YOLO26n and YOLO26s.

The increase in validation mAP@50-95 from 0.3638 to 0.3983 is an absolute improvement of 0.0345 (about 9.5% relative to the baseline). YOLO26s also increased mAP@50 by about 0.0995 and Recall by about 0.2287. Training time increased from 28.50 to 31.55 minutes (about 10.7%), which is a modest cost for the accuracy gain. The measured validation inference time was not worse for YOLO26s in this run (11.20 ms/image versus 12.47 ms/image), but this timing difference should not be over-interpreted because runtime measurements can vary with warm-up, caching and system load.

The comparison supports choosing YOLO26s for this dataset because the project objective favours finding and localising labelled defects rather than maximising Precision alone. However, each candidate model was trained only once with seed 42, so small differences may include random training variation.

6. Final Test, Sensitivity and Engineering Interpretation

After model selection, YOLO26s was evaluated on the withheld test split. The final test results were Precision = 0.4394, Recall = 0.6513, mAP@50 = 0.5902 and mAP@50-95 = 0.2776. The reduction from validation mAP@50-95 (0.3983) to test mAP@50-95 (0.2776) shows that generalisation remains limited. The experiment does not isolate a single cause, but the severe class imbalance, small test set and very low support for rare classes are credible contributors.

Image size	mAP@50-95	Inference ms/image
512	0.3855	6.98
640	0.3983	8.74

Table 4. Sensitivity of the selected model to input image size on validation data.

Reducing the input size from 640 to 512 lowered validation mAP@50-95 from 0.3983 to 0.3855 but reduced inference time from 8.74 to 6.98 ms/image. This demonstrates a simple engineering trade-off between accuracy and speed. The final configuration keeps 640 pixels because the small accuracy improvement was considered more important than the additional inference time for this project.

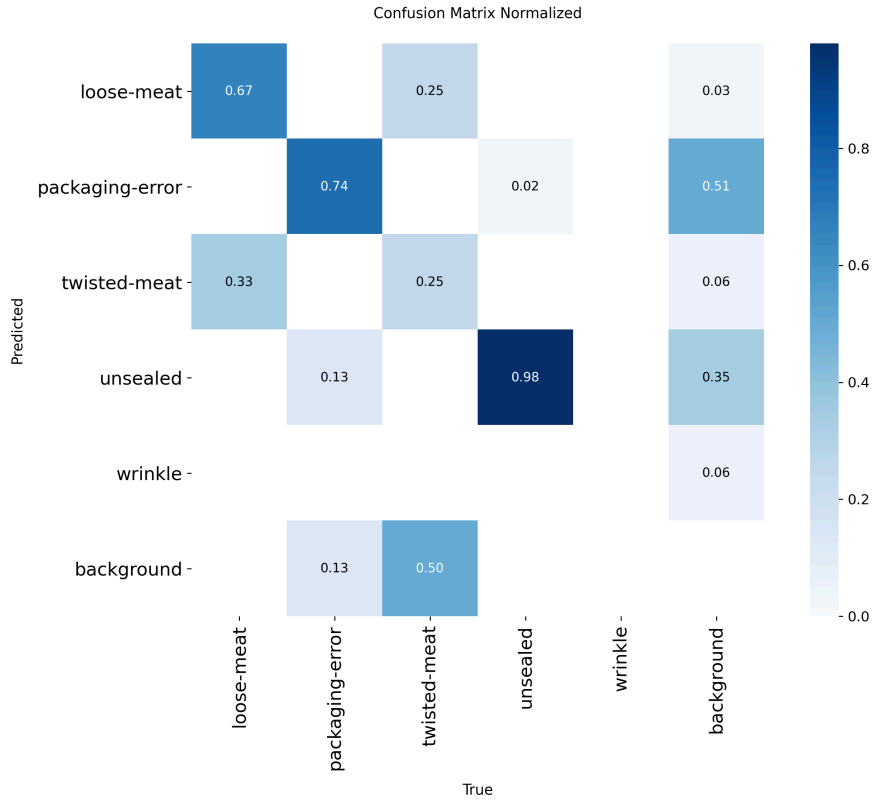


Figure 3. Normalised confusion matrix for the final test evaluation. The wrinkle column has no test ground-truth examples.

The confusion matrix indicates that unsealed is the most consistently recognised supported class in the small test set, while the rare classes are less stable. Background-related false detections are also visible. In an industrial setting, these errors would matter because unnecessary alarms increase manual review, while missed defects reduce inspection effectiveness. The current results therefore support a proof-of-concept detector rather than a production-ready inspection system.

7. Limitations and Lessons Learned

- The dataset is highly imbalanced. Approximately 94.5% of all annotations belong to packaging-error or unsealed, while wrinkle represents only about 0.6%.
- Wrinkle has no validation or test instances, so its detection performance is not validated. Loose-meat and twisted-meat also have very small validation/test support.
- There are 710 empty-label images. They are retained because they may be legitimate no-defect/background examples, but their meaning should be confirmed from the dataset creation process.
- Only one training run per model was completed using seed 42. Repeating runs with several seeds would give a better estimate of repeatability.
- The final test mAP@50-95 is 0.2776, so localisation quality is not yet strong enough to justify production deployment.
- The model detects visual classes only. It does not measure seal strength, temperature, contamination, shelf life or any other physical quality property.
- The experiment used one provided dataset. Performance may change with another camera, lighting condition, conveyor setup, packaging material or product presentation.

The main lesson from the comparison is that a larger model is not automatically better in every metric. YOLO26s improved Recall and mAP, but its Precision decreased and training took longer. Model selection therefore depends on the chosen engineering objective and should be based on measured trade-offs rather than model size alone.

A second lesson is that dataset quality and class support can be more important than further model tuning. The absence of wrinkle examples in validation and test prevents meaningful verification for that class. A stronger next step would be to collect or rebalance rare-class examples, then repeat validation before attempting more complex architectures.

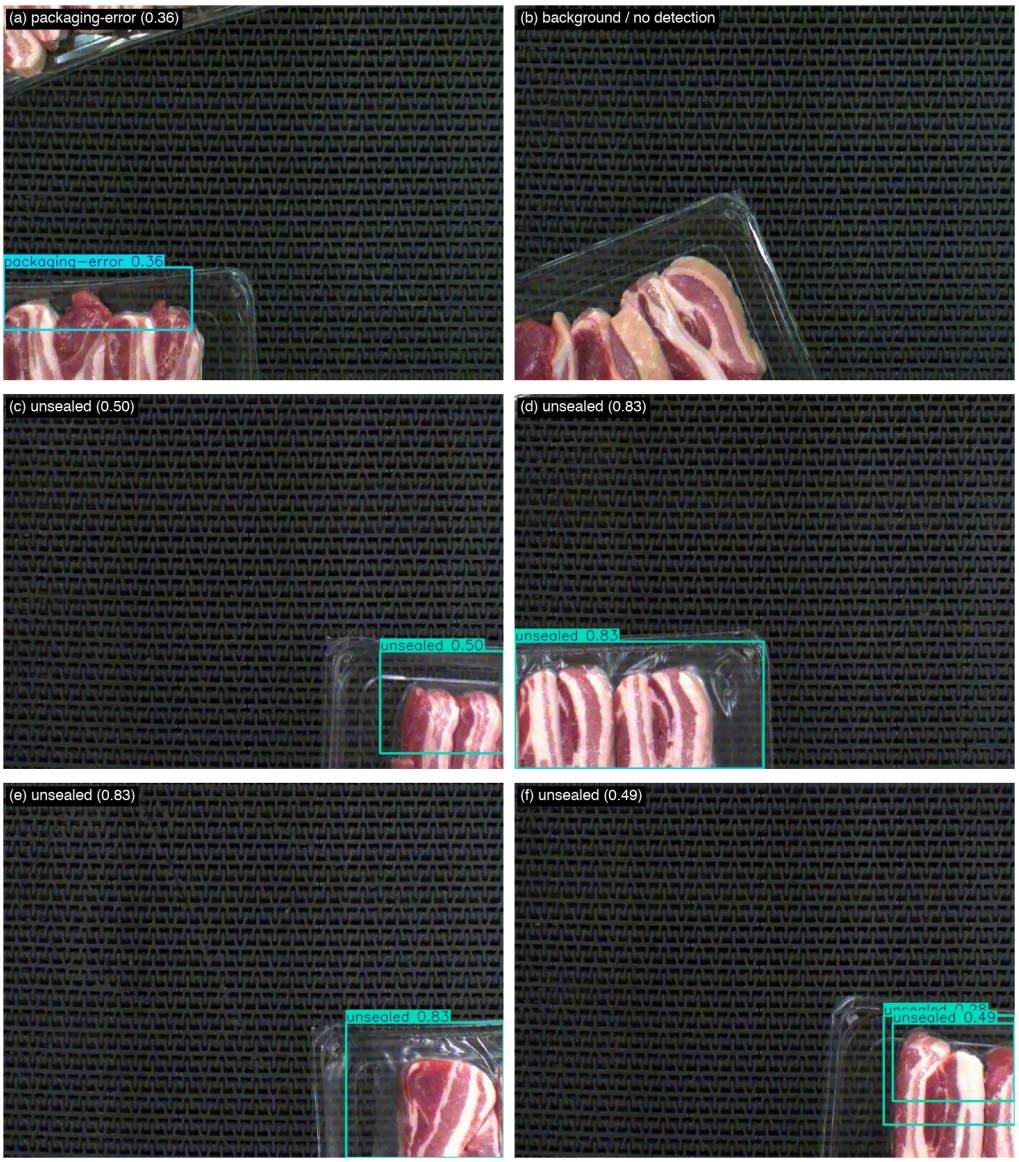


Figure 4. Example final-model predictions on test images. Boxes show predicted defect locations and labels.

8. AI Use Reflection

This project was independently designed, implemented, evaluated, and documented by the student. AI tools (primarily ChatGPT) were utilised strictly in an auxiliary and consultative capacity, serving as a secondary reference for debugging runtime errors, verifying library syntax, and receiving organizational feedback on repository structuring. The core engineering work, model development, data auditing, and written analysis were conducted entirely by the student.

AI was engaged selectively during development to troubleshoot technical hurdles, such as resolving Colab environment paths and diagnosing Ultralytics configuration flags. All suggestions and diagnostic outputs were critically evaluated and verified against the actual dataset and codebase before adoption. In several instances, AI proposed overly elaborate architectures or redundant profiling scripts; these were actively filtered and discarded in favour of a clean, robust, and reproducible baseline-versus-alternative workflow.

All principal engineering decisions remained strictly under the student's authority: formulating the problem scope, selecting the candidate models (YOLO26n and YOLO26s), establishing validation mAP@50-95 as the primary selection criterion, executing training on Colab, evaluating the input-size sensitivity, auditing the 710 negative background samples, and critically analysing the severe class imbalance and absence of validation support for the wrinkle class. AI acted solely as an interactive advisory tool, ensuring full student ownership and methodological integrity.

9. Conclusion

This project developed a reproducible object-detection workflow for visible frozen-meat packaging defects. Two YOLO26 model sizes were compared using the same main training configuration. YOLO26s was selected because it achieved the higher validation mAP@50-95 (0.3983 versus 0.3638) and higher Recall. On the withheld test set, the selected model achieved Precision 0.4394, Recall 0.6513, mAP@50 0.5902 and mAP@50-95 0.2776.

The sensitivity check showed that 512-pixel input reduced inference time but also slightly reduced validation accuracy, so 640 pixels was retained. The project demonstrates a functioning machine-vision proof of concept, but the results also show that dataset imbalance and weak rare-class support limit confidence in the system. The most useful future improvement is therefore to obtain more balanced validation and test data, especially for wrinkle, loose-meat and twisted-meat, before increasing model complexity.

References

[1] MMA3001, “Numerical Methods and Machine Learning - Project Brief,” 2026.

[2] Roboflow, “Pork Rasher Error Packaging,” dataset version 4, Roboflow Universe. Dataset metadata records CC BY 4.0 licensing. https://universe.roboflow.com/hello-2aqe0/pork-rasher-error-packaging/dataset/4

[3] Ultralytics, “Ultralytics YOLO,” software package used for YOLO26 training, validation and prediction. Experiment environment: Ultralytics 8.4.173.

[4] PyTorch, “PyTorch,” machine-learning framework used by the Ultralytics training environment. Experiment environment: PyTorch 2.11.0 with CUDA on a Tesla T4 GPU.

Appendix A. Repository Evidence

The final project package contains the standalone Jupyter notebook, Python source utilities with docstrings, pytest unit tests, an HTML test report, HTML API documentation, the selected model weights, measured CSV/TXT outputs, confusion matrices and example predictions. The GitHub upload and presentation are intentionally left for the final submission stage.